

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		Assignment Type: Lab	AcademicYear: 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator)	
		Dr. T. Sampath Kumar	
		Dr. Pramoda Patro	
		Dr. Brij Kishor Tiwari	
		Dr. J. Ravichander	
		Dr. Mohammand Ali Shaik	
		Dr. Anirodh Kumar	
		Mr. S. Naresh Kumar	
		Dr. RAJESH VELPULA	
		Mr. Kundhan Kumar	
		Ms. Ch. Rajitha	
		Mr. M Prakash	
		Mr. B. Raju	
		Intern 1 (Dharma teja)	
		Intern 2 (Sai Prasad)	
		Intern 3 (Sowmya)	
		NS_2 (Mounika)	
CourseCode	24CS002PC215	CourseTitle	AI Assisted Coding
Year/Sem	II/I	Regulation	R24
Date and Day of Assignment	Week1 - Wednesday	Time(s)	
Duration	2 Hours	Applicable to Batches	24CSBTB01 To 24CSBTB39
AssignmentNumber: 1.3 (Present assignment number) / 24 (Total number of assignments)			
Q.No.	Question	Expected Time to complete	
1	Lab 1: Environment Setup – GitHub Copilot and VS Code Integration Lab Objectives: To install and configure GitHub Copilot in Visual Studio Code.	Week1 - Wednesday	

- To explore AI-assisted code generation using GitHub Copilot.
- To analyze the accuracy and effectiveness of Copilot's code suggestions.
- To understand prompt-based programming using comments and code context

Lab Outcomes (LOs):

After completing this lab, students will be able to:

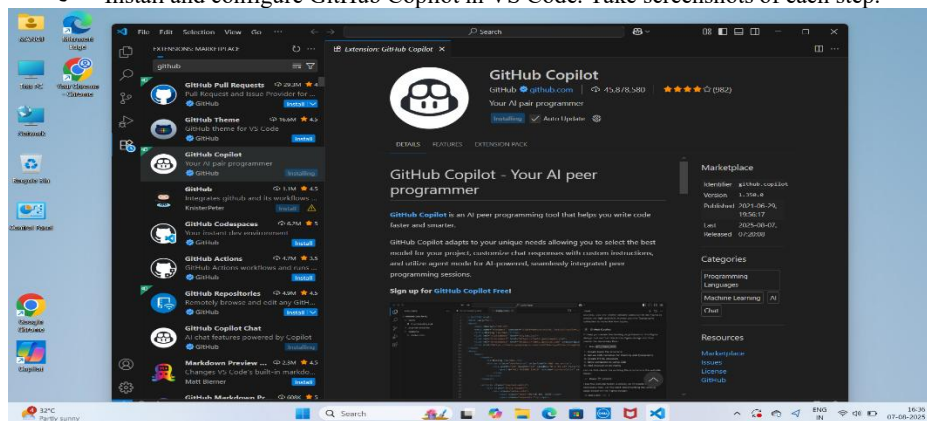
- Set up GitHub Copilot in VS Code successfully.
- Use inline comments and context to generate code with Copilot.
- Evaluate AI-generated code for correctness and readability.
- Compare code suggestions based on different prompts and programming styles.

Task Description#1

- Install and configure GitHub Copilot in VS Code. Take screenshots of each step.

Expected Output#1

- Install and configure GitHub Copilot in VS Code. Take screenshots of each step.

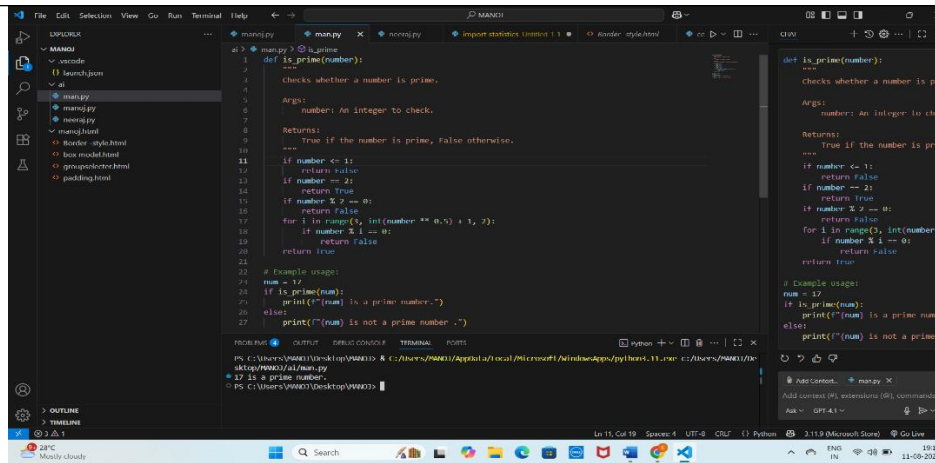


Task Description#2

- Use Copilot to generate a `is_prime()` Python function.

Expected Output#2

- Function to check primality with correct logic.
-

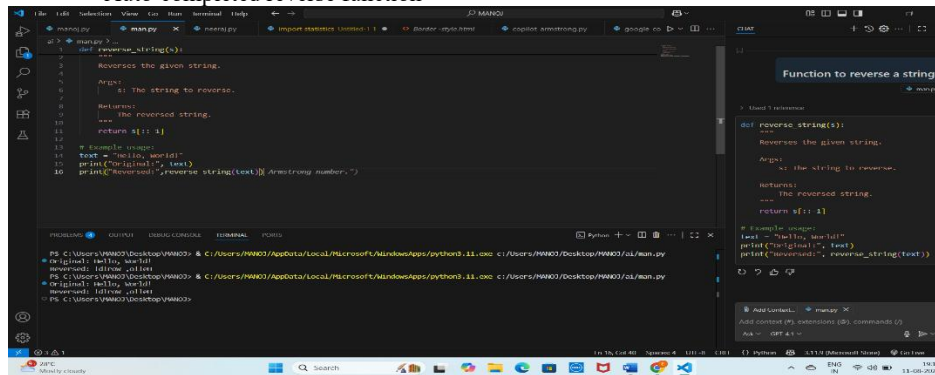


Task Description#3

- Write a comment like # Function to reverse a string and use Copilot to generate the function.

Expected Output#3

- Auto-completed reverse function

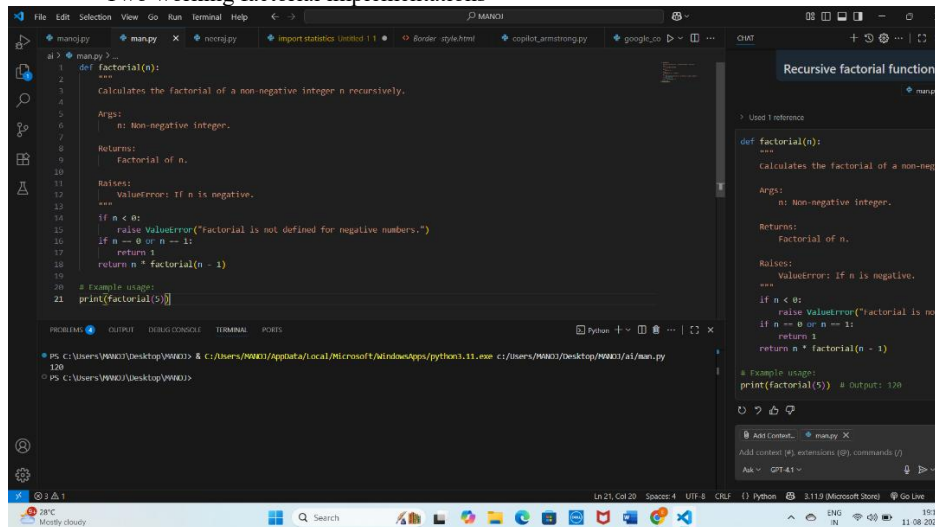


Task Description#4

- Generate both recursive and iterative versions of a factorial function using comments..

Expected Output#4

- Two working factorial implementations



```
1 def factorial_iterative(n):
2     """
3     Calculates the factorial of a non-negative integer n.
4     """
5     if n < 0:
6         raise ValueError("factorial is not defined for negative numbers.")
7     result = 1
8     for i in range(2, n + 1):
9         result *= i
10    return result
11
12 # Test case for Armstrong number
13 def is_armstrong_number(n):
14     """
15     Checks if a number is an Armstrong number.
16     """
17     if n < 0:
18         raise ValueError("factorial is not defined for negative numbers.")
19     result = 1
20     for i in range(2, n + 1):
21         result *= i
22     return result
23
24 # Example usage
25 print(factorial_iterative(5)) # output: 120
26 print(factorial_iterative(153)) # output: 120
```

Task Description#5

- Use Copilot to find the largest number in a list. Assess code quality and efficiency.

Expected Output#5A a valid function with your review

```
1 def find_largest_number(numbers):
2     """
3     Finds the largest number in a list.
4     """
5     Args:
6         numbers: A list of numbers.
7
8     Returns:
9         The largest number in the list, or None if the list is empty.
10    """
11    if not numbers:
12        return None
13    return max(numbers)
14
15 # Example usage
16 num_list = [3, 7, 2, 9, 5]
17 print("largest number:", find_largest_number(num_list))
```

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots